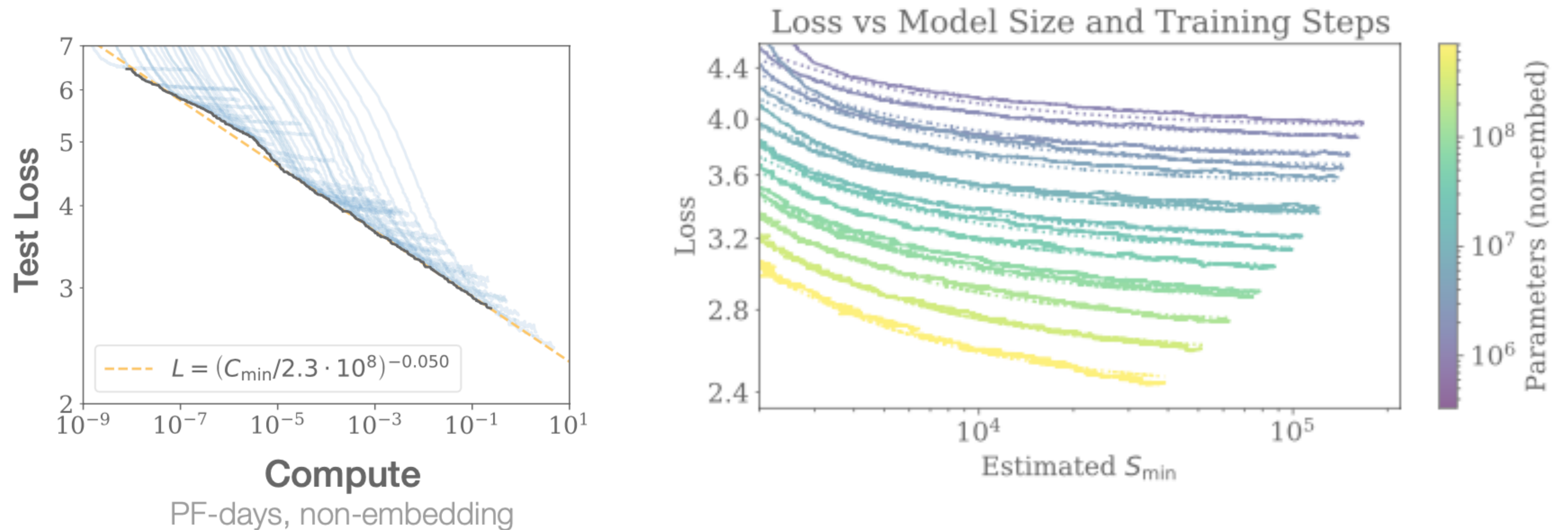


Lecture 12: Foundation Models II (of 3)

Scaling Laws and Emergent Behaviors

Speaker: Omar Khattab



Outline

1. Evaluation
2. Scaling Laws
3. Beyond Scaling Laws

Lecture 15: Post-Training & RL

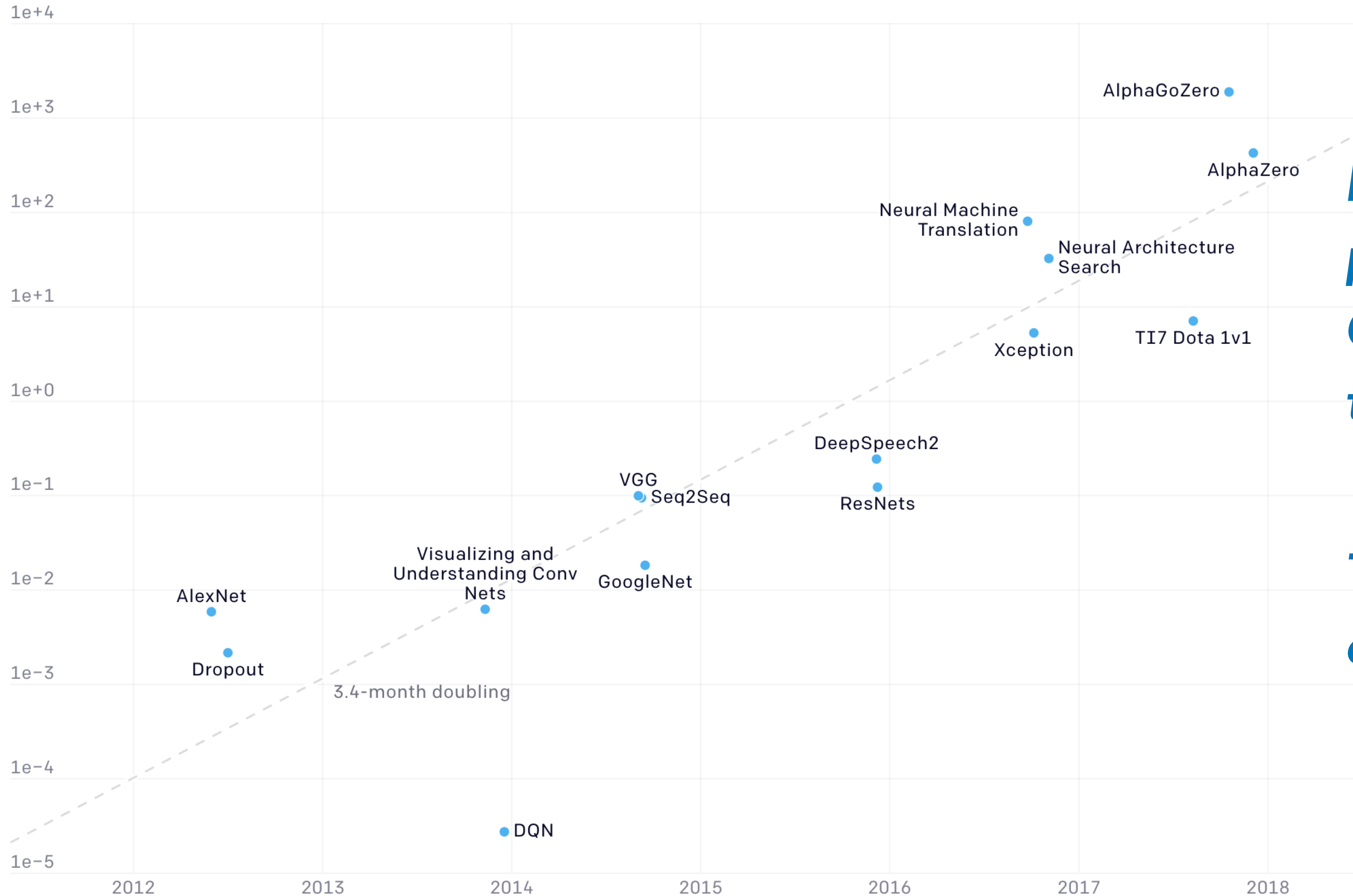
Lecture 21: Inference-time systems

Last lecture, we motivated **foundation models**...

- ... via self-supervised pre-training, for next-token prediction
- ... for decoder-only Transformers, trained on somewhat curated but massively scaled Web data.
- Can result in learning lots of knowledge, skills, ... if scaled.
- And people have been scaling this to incredible degrees!

AlexNet to AlphaGo Zero: A 300,000x Increase in Compute (Log Scale)

Petaflop/s-days



In 2018, in the pre-BERT era, OpenAI published this plot.

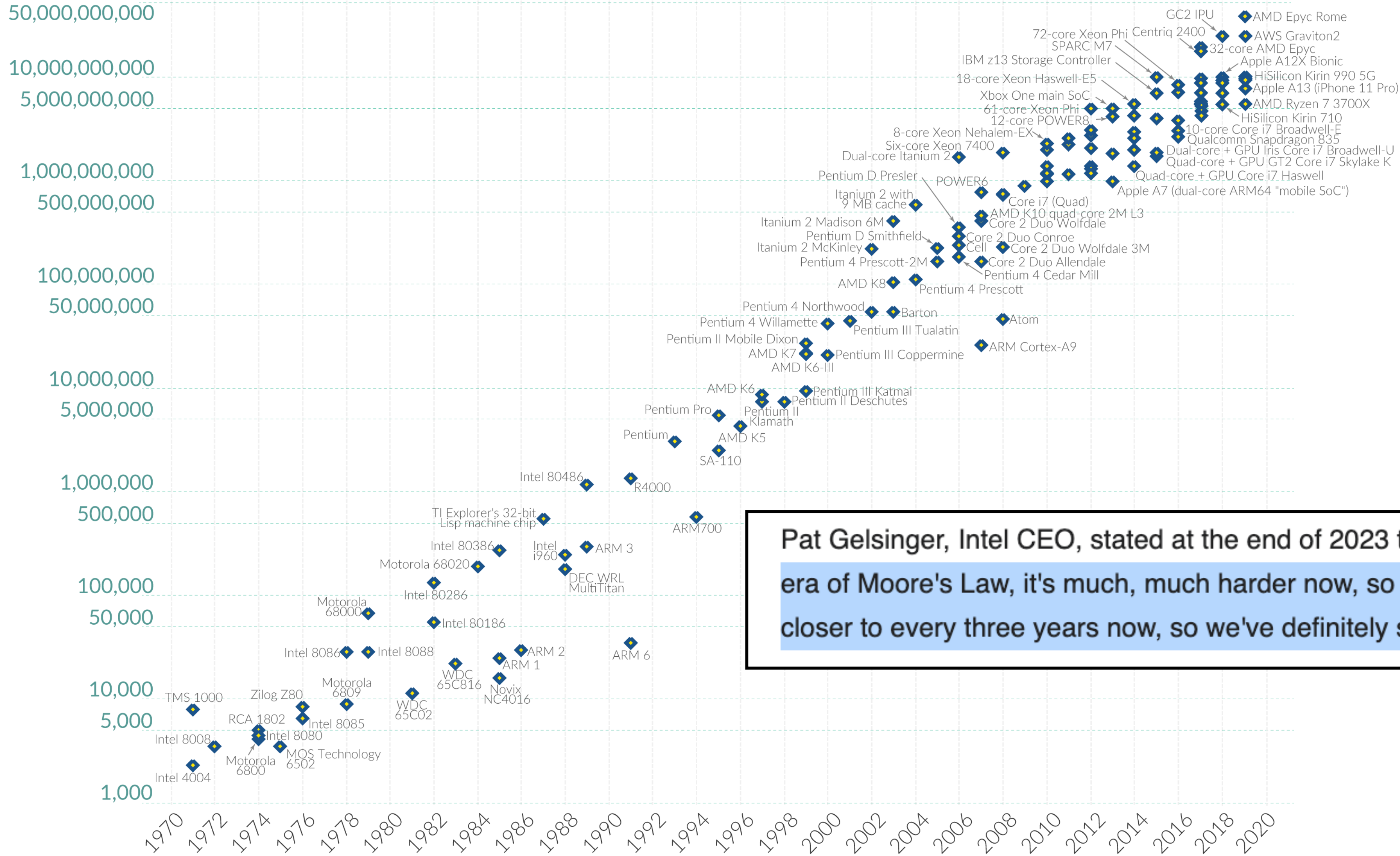
The trends have continued since!

Moore's Law: The number of transistors on microchips doubles every two years

Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Transistor count



An obsession with scaling is not unique to ML. Characteristic of EE and CS.

The Bitter Lesson

Rich Sutton

March 13, 2019

The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore's law, or rather its generalization of continued exponentially falling cost per unit of computation. Most AI research has been conducted as if the computation available to the agent were constant (in which case leveraging human knowledge would be one of the only ways to improve performance) but, over a slightly longer time than a typical research project, massively more computation inevitably becomes available. Seeking an improvement that makes a difference in the shorter term, researchers seek to leverage their human knowledge of the domain, but the only thing that matters in the long run is the leveraging of computation. These two need not run counter to each other, but in practice they tend to. Time spent on one is time not spent on the other. There are psychological commitments to investment in one approach or the other. And the human-knowledge approach tends to complicate methods in ways that make them less suited to taking advantage of general methods leveraging computation. There were many examples of AI researchers' belated learning of this bitter lesson, and it is instructive to review some of the most prominent.

Evaluation

Validation: What do you keep an eye on during training?

Test: How do we compare different LLMs quantitatively?

How do you know any of this is any good?

- Loss on pre-training validation set (avg cross-entropy across tokens)

$$\mathcal{L} = \frac{1}{N} \sum_{t=1}^N -\log p_{\theta}(x_t | x_{<t})$$

- Perplexity (PPL) = $e^{\mathcal{L}}$

Intuition: “On avg, the model is as uncertain as selecting between PPL equally-likely options.”

What’s attractive about this? (Universal & smooth!)

	LAMBADA (PPL)	LAMBADA (ACC)	CBT-CN (ACC)	CBT-NE (ACC)	WikiText2 (PPL)	PTB (PPL)	enwik8 (BPB)	text8 (BPC)	WikiText103 (PPL)	1BW (PPL)
SOTA	99.8	59.23	85.7	82.3	39.14	46.54	0.99	1.08	18.3	21.8
117M	35.13	45.99	87.65	83.4	29.41	65.85	1.16	1.17	37.50	75.20
345M	15.60	55.48	92.35	87.1	22.76	47.33	1.01	1.06	26.37	55.72
762M	10.87	60.12	93.45	88.0	19.93	40.31	0.97	1.02	22.05	44.575
1542M	8.63	63.24	93.30	89.05	18.34	35.76	0.93	0.98	17.48	42.16

Category	Representative Dataset	Example
Cloze tasks (LM-like but selective)	LAMBADA ('16)	"Preston had been the last person to wear those chains. [...] But he put down the ____" (chains)
General Language Understanding	SuperGLUE ('19)	Is windows movie maker part of windows essentials? (Yes/No)
General Knowledge & Problem Solving	MMLU ('20)	what is the agenda for hollande's state visit to washington
Instruction Following	IFEval ('23)	can you make and receive calls in airplane mode
"Deep Research"	HotPotQA('18), HoVer ('20), and BrowseComp ('25)	"Which MVP of a World Series game in which Red Flaherty umpired was later elected to the Baseball Hall of Fame?"
Code Generation	HumanEval ('21), SWE-Bench ('23)	"Write a Python snippet implementing: From a list of integers, remove all elements that occur more than once"
Computer Use	OSWorld ('24), AppWorld ('24)	"Approve all Vennmo payment requests from my roommates from this month."
Live Human Usage!	Chatbot Arena (2023–)	<i>People ask hard questions, e.g. tricky coding problems or personal assistant requests, and vote on anonymous LLMs!</i>

ML is in a deep evals crisis...

Contamination. How do you ensure benchmarks are **not** in your pre-training data?

Explicit hill-climbing. By '23, LLMs switched from “few-shot learners” to aggressive training & validation on all benchmarks. (Sometimes test sets, too! Hopefully at least transparently!)

Emergence. Capabilities “emerge” from lots of data. This often makes it easier to build a system for task T than to reliably evaluate capability T!

As we scaled up the project, we began having to collect labels on multiple solutions for the same training problem. In order to avoid the risk of over-fitting on the 7,500 MATH training problems, we expanded the training set to include 4,500 MATH test split problems. We therefore evaluate our models only on the remaining 500 held-out problems. We selected these 500 test problems uniformly at random. In Figure 5, we show that the distribution of difficulty levels and subjects in this subset is representative of the MATH test set as a whole. The specific test set we used can be found at <https://github.com/openai/prm800k>.

Scaling Laws

If you had a given budget of compute, what model would you train on how much data?

So many decisions to make...

- I have a fixed GPU budget. Make model larger? Or crawl more data?
- **Data:** Which of several different data mixtures do I use?
- **Architecture:** Do I train a Transformer, a *modified* Transformer, an LSTM?
- Other hyperparams too, like optimizer, depth/width, learning rate, ...
- Idea #1: Largest model!
 - Train 50 models and pick best on dev! *Challenges?*
 - Then you can only train with 1/50 of your budget...
 - Are 50 runs even enough to cover your different choices?

Idea #2: Tune hyperparams at small scale!

- Then just copy them over and run them at large scale.
- *Challenge?*
- ... turns out they don't transfer so easily! Best choice at small scale may not be the same at larger scale. (E.g., inductive biases or LR)
- This also doesn't tell you how large to make your choices.

Idea #3: Study “Scaling Laws”

- Train and tune many **smaller models** (starting from what choices?)
- Plot your loss vs. FLOPs (cost) as you scale up, on a log-log plot.
- *Extrapolate, for each training scheme??*
- *Given a target loss, find the “cheapest” way to get it!*

Learning Curves: Asymptotic Values and Rate of Convergence

Corinna Cortes, L. D. Jackel, Sara A. Solla, Vladimir Vapnik,
and John S. Denker
AT&T Bell Laboratories
Holmdel, NJ 07733

Training classifiers on large databases is computationally demanding. It is desirable to develop efficient procedures for a reliable prediction of a classifier’s suitability for implementing a given task, so that resources can be assigned to the most promising candidates or freed for exploring new classifier candidates. We propose such

Scaling laws

minimize *test-loss(model size, data, batch size, steps,...)* **such that** *compute* **within budget**

for a given architecture, how much data / params would be needed for a target performance?

can we extrapolate from smaller to larger models / experiments??

$$N_{opt}(C), D_{opt}(C) = \underset{N,D \text{ s.t. } \text{FLOPs}(N,D)=C}{\operatorname{argmin}} L(N, D)$$

How does the **test loss** scale with *data* and *parameters*?

- decoder-only Transformer ("GPT")
- data: WebText2, 96GB of text, $2.29 \cdot 10^{10}$ tokens
1024 token context
- vary model size: 768 - 1.5 billion non-embedding parameters
- and data size: 22M - 23B tokens, among other axes
- compute = parameters x batch x steps

Scaling Laws for Neural Language Models

Jared Kaplan *

Johns Hopkins University, OpenAI

jaredk@jhu.edu

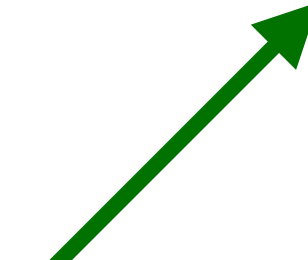
Sam McCandlish*

OpenAI

sam@openai.com

Key Finding: **power law** relationships

$$L(X) = (1/X)^\alpha = X^{-\alpha}$$

test loss  **resource**
**(data/parameters/
compute)** 

*But what's the
value of alpha?*

*Varies with your
pre-training design.*

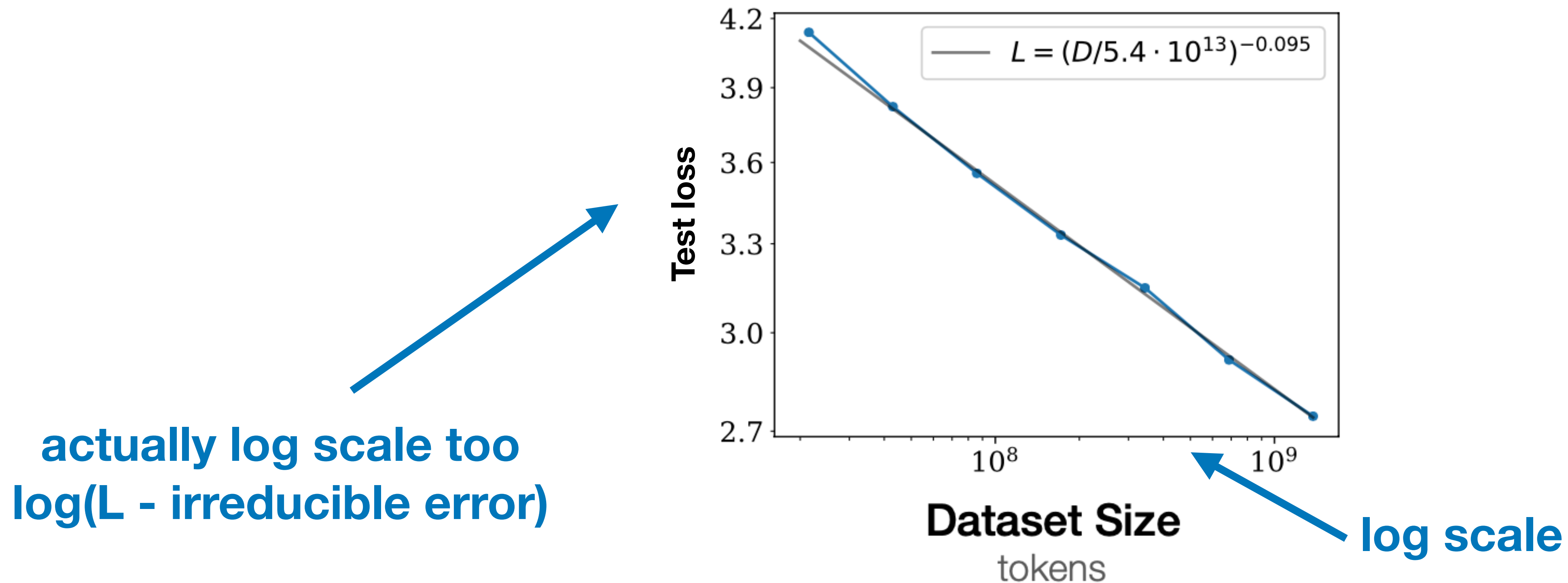
- scale $X \rightarrow 2X$ $\Rightarrow$ Loss $\rightarrow 2^{-\alpha}$ Loss

***We want to know
not how things
perform at small
sizes, but how well
they scale.***



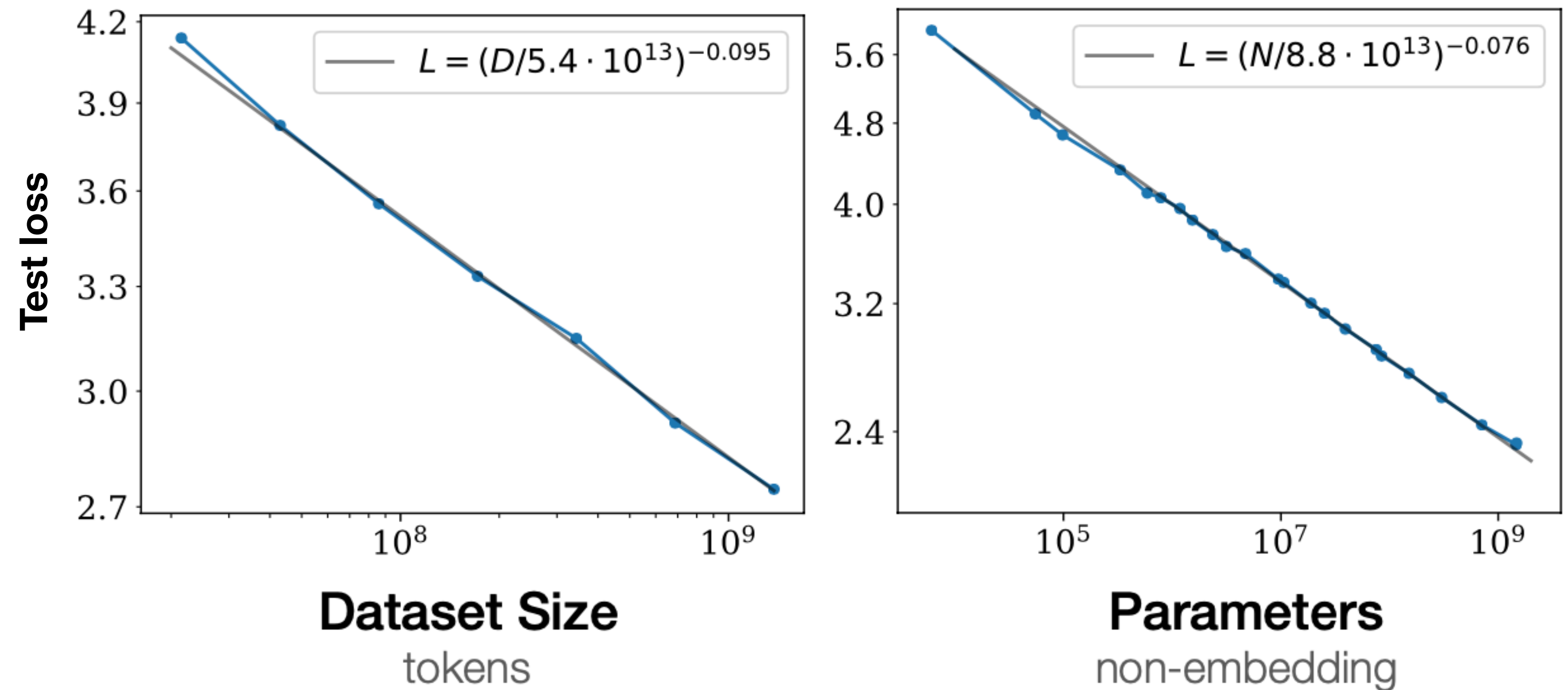
How does the test loss scale **as a function of data size**?

If *parameters* and total compute are too large to be bottlenecks (“infinite”)...



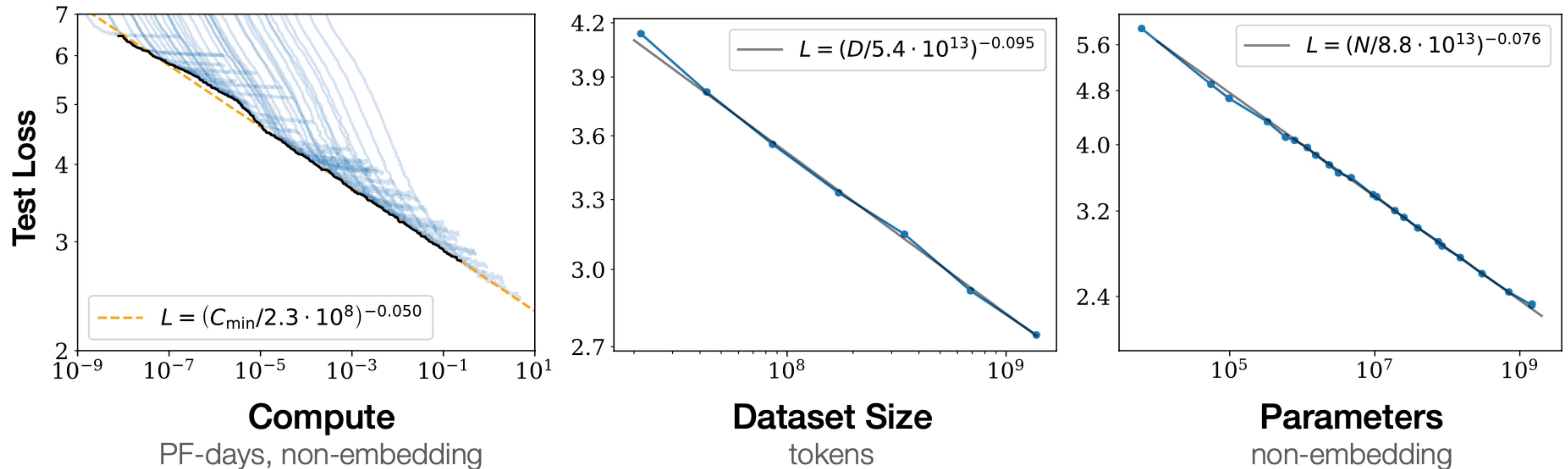
How does the test loss scale with **number of parameters**?

If *tokens* and total compute are too large to be bottlenecks (“infinite”)...



How does the test loss scale with **available compute**?

- power law too, if you allocate compute “optimally”



Takeaways of *Kaplan et al.*

- test loss scales as **power law** of data, parameter count, and compute
- test loss is more sensitive to parameter count than data: with more compute, **increase model by a larger factor than dataset size (!)**
- larger models require less data to reach same performance, so don't train to convergence — instead: train larger models for fewer steps

***This popularized a very different and new way of thinking.
Is model (# of parameters) scale pretty much all you need??***

(No, but it was a very useful illusion in some sense!)

Scaling Laws caveats... Kaplan'20 vs. Chinchilla'22

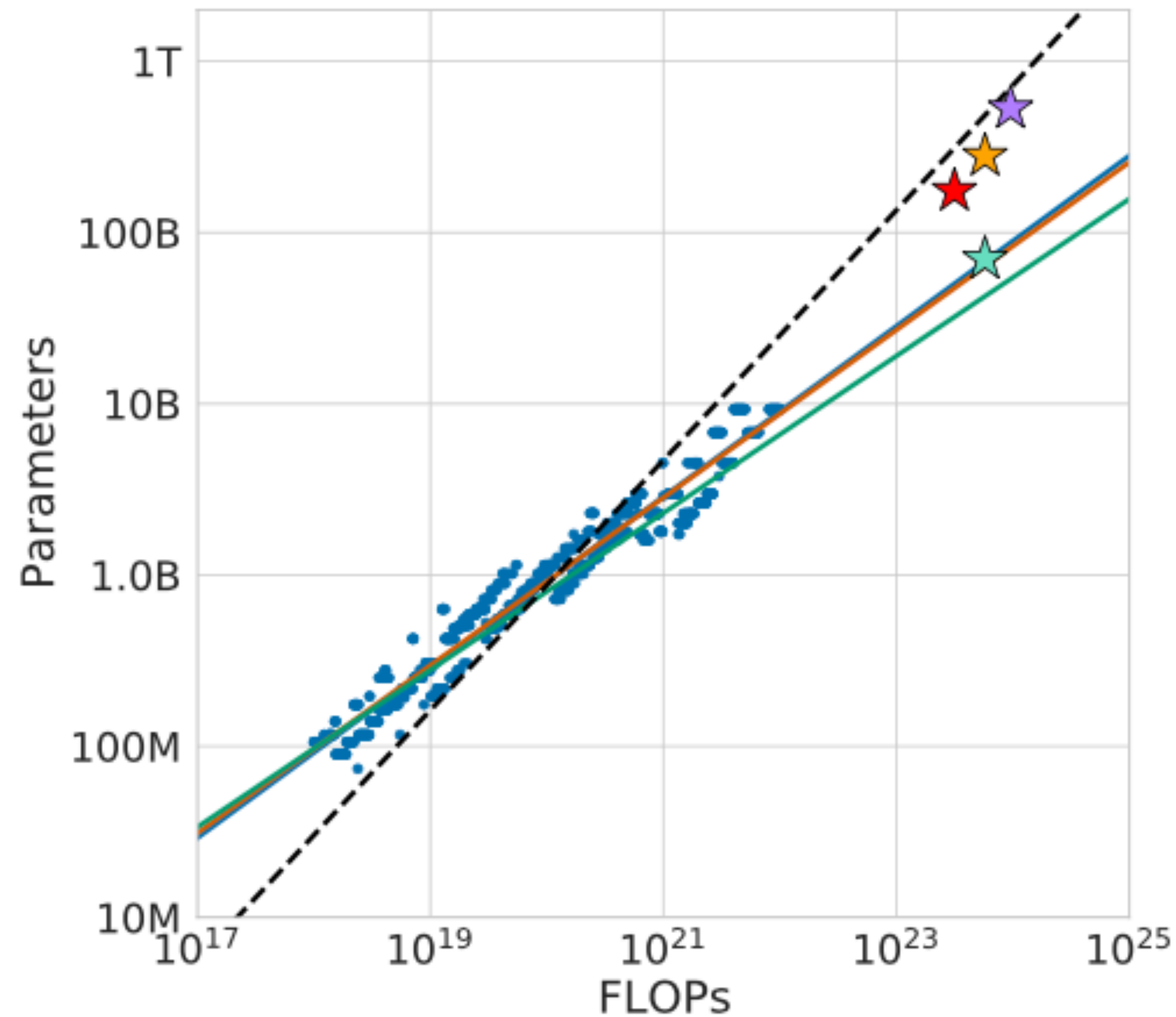
DeepMind confirmed the power-law scaling but disputed the optimal allocation between data and parameters. Scale them equally! (sqrt of compute)

Details: They fixed some hyperparameter decisions (data-dependent learning rate schedule) and experimented with much larger models (up to 16B in scale). They also counted parameters a little differently (embedding parameters counted).

“Fixed training budget” vs. deployment.
There's a shift towards *overtraining*!

- **GPT3** – 2 tokens / param
- **Chinchilla** – 20 tokens / param
- **LLaMA65B** – 22 tokens / param
- **Llama 2 70B** – 29 tokens / param
- **Mistral 7B** – 110 tokens / param
- **Llama 3 70B** – 215 tokens / param

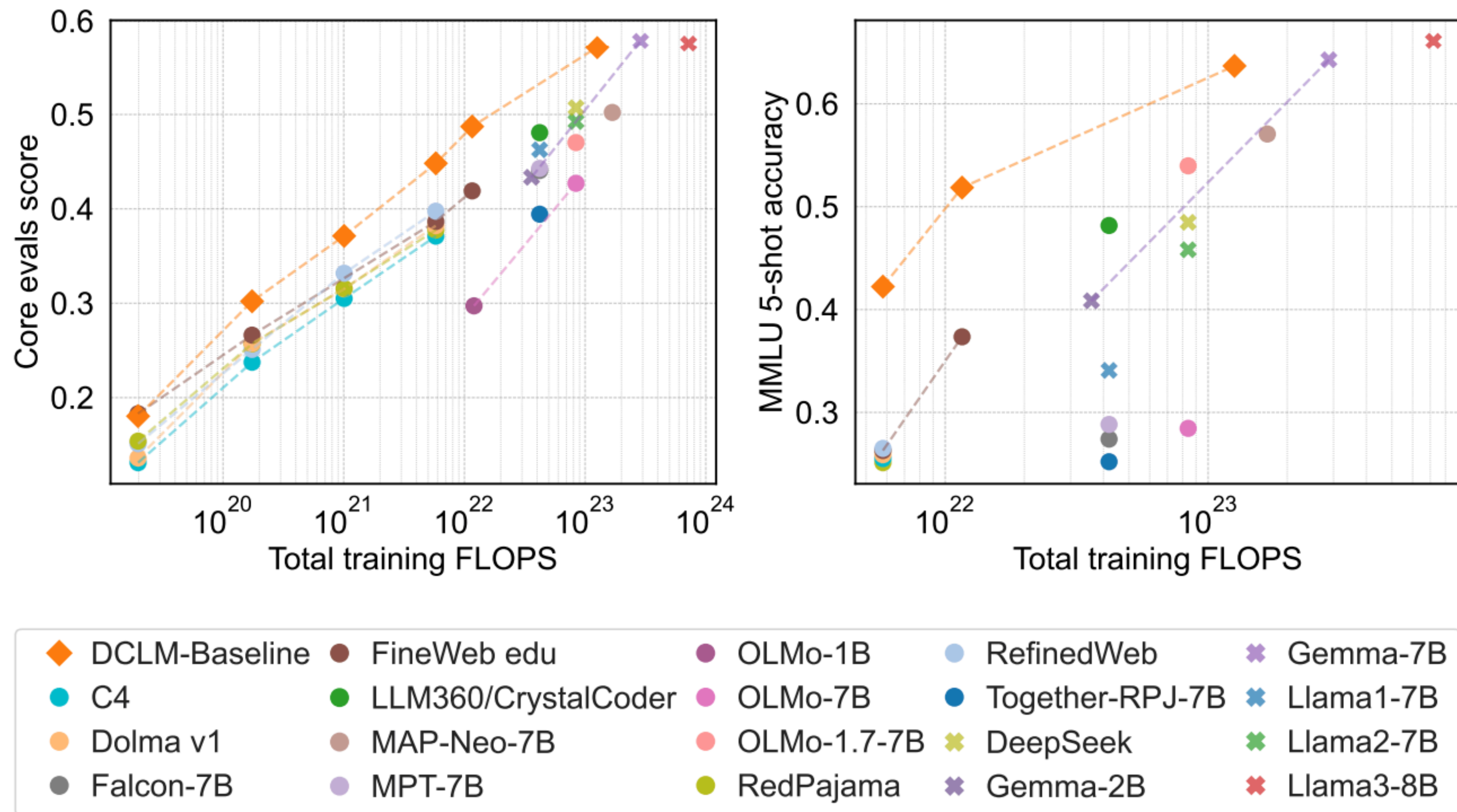
Predictions: extrapolation



For a given #FLOPs, how many parameters should we allocate?
(compute = 6 x data x params)

- Approach 1
 - Approach 2
 - Approach 3
 - Kaplan et al (2020)
-
- Chinchilla (70B)
 - Gopher (280B)
 - GPT-3 (175B)
 - Megatron-Turing NLG (530B)

Related: How do we know **what data to use**?



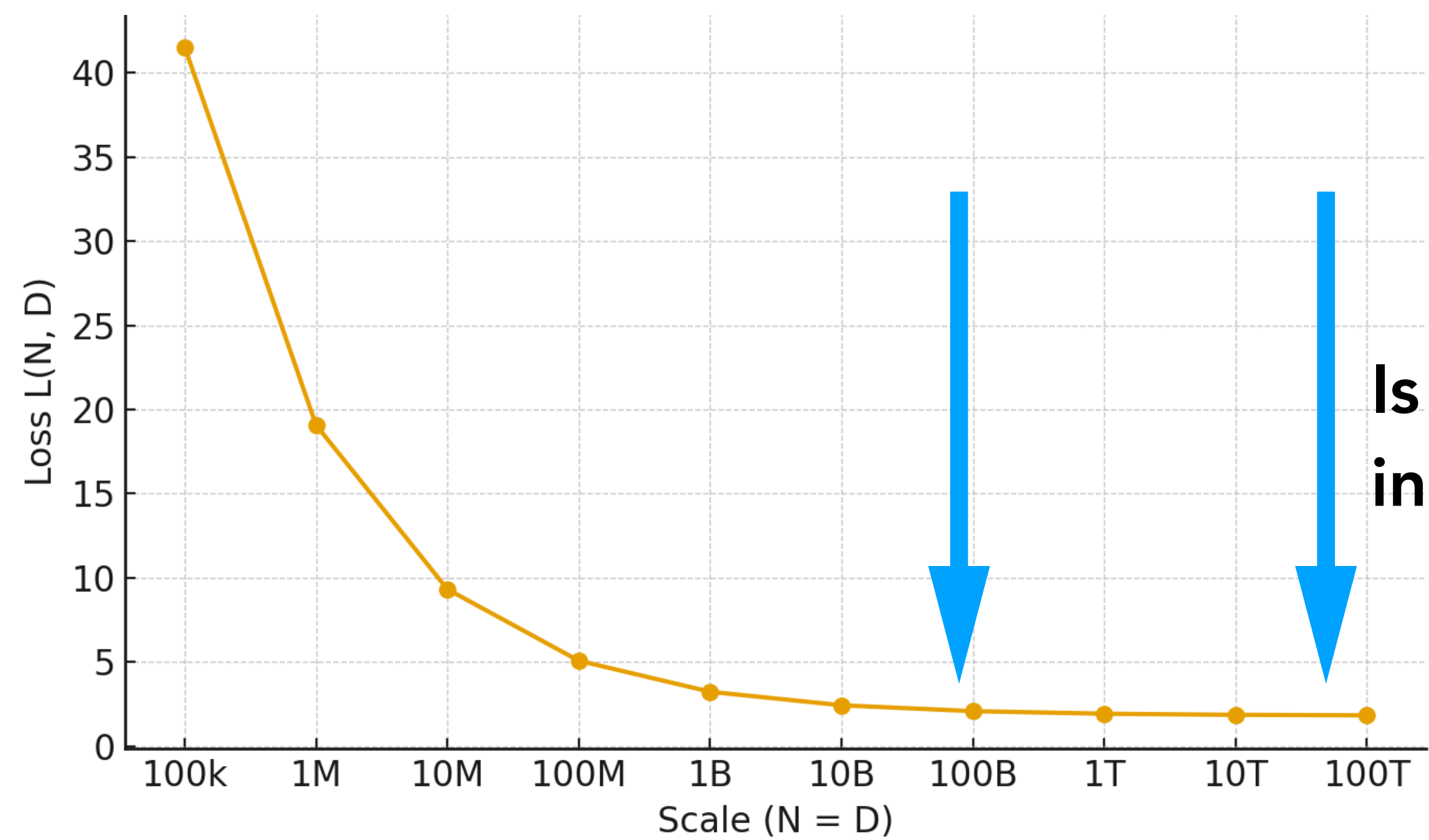
Beyond Scaling Laws

Suppose scaling laws hold indefinitely. Would be awesome, right?

A power law means **diminishing returns**.

$$L(N, D) = 1.8172 + \frac{482.01}{N^{0.3478}} + \frac{2085.43}{D^{0.3658}}$$

Chinchilla Scaling: A replication attempt (2024)



N (or D)	L(N, D)
100k	41.525437
1M	19.080769
10M	9.324990
100M	5.083290
1B	3.238487
10B	2.435894
100B	2.086609
1T	1.934553
10T	1.868336
100T	1.839490

What does reducing the loss actually “mean”?

- Should we fixate so much on perplexity?
- For a fixed architecture, it tends to correlate broadly with downstream quality — on *most* tasks.
- But is increasingly a weak proxy for capability, reasoning, safety, ...
- How do skills we're interested in behave and scale?

Scaling appears to create **emergent behaviors**

- *Sudden jumps* in capability as we increase parameters (or compute)
- Includes the ability to do “few-shot” **in-context learning**
- Refers to the ability of a pre-trained LLM to solve tasks given nothing but a **task description and possibly a few examples in its context**

Traditionally, DNNs would require **task fine-tuning**

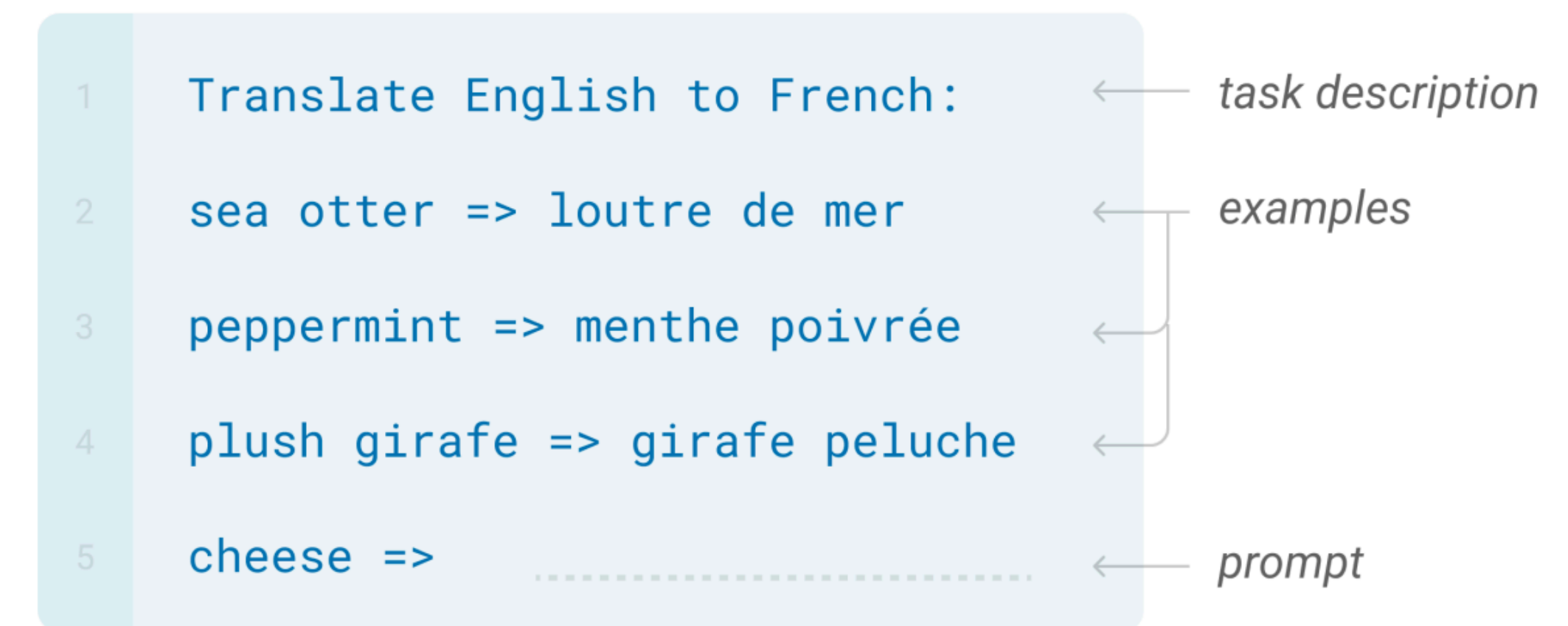
Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Hard to overstate how surprising this was from vanilla pre-training.

After decades of research on meta learning, this ability “emerges” from scale and next-token prediction!

Q: Who was president of the United States in 1955?

A: Dwight D. Eisenhower was president of the United States in 1955.

Q: Which party did he belong to?

A: He belonged to the Republican Party.

Q: What is the square root of banana?

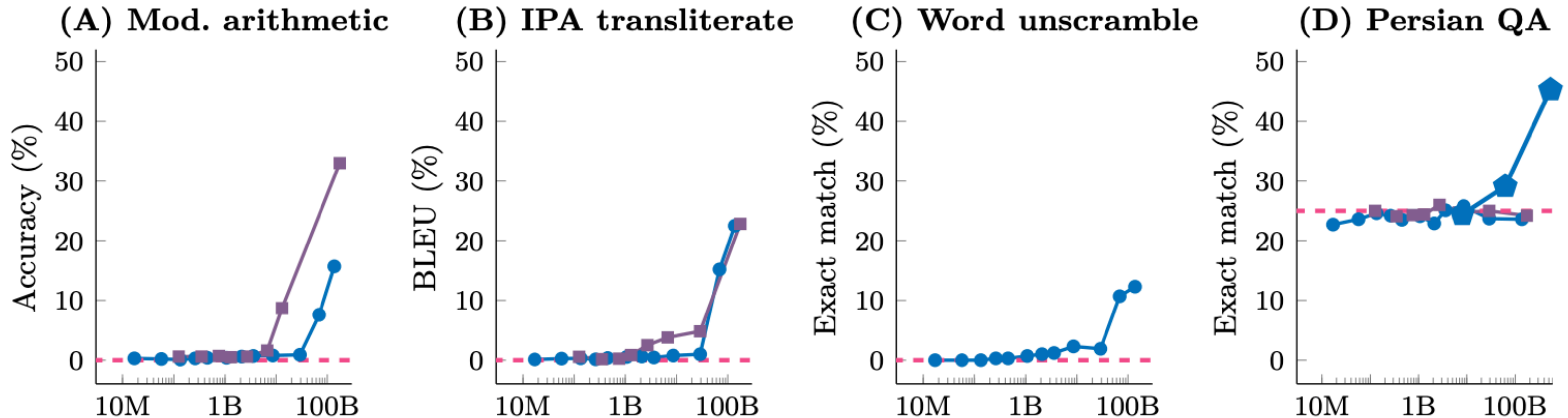
A: Unknown

Q: How many wards does Tokyo consist of?

A: Tokyo consists of 23 wards.

Q:|

In-context learning as an emergent behavior



Thus, emergent abilities cannot be predicted simply by extrapolating the performance of smaller models. The existence of such emergence raises the question of whether additional scaling could potentially further expand the range of capabilities of language models.

Chain-of-thought 'reasoning' as an emergent behavior

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

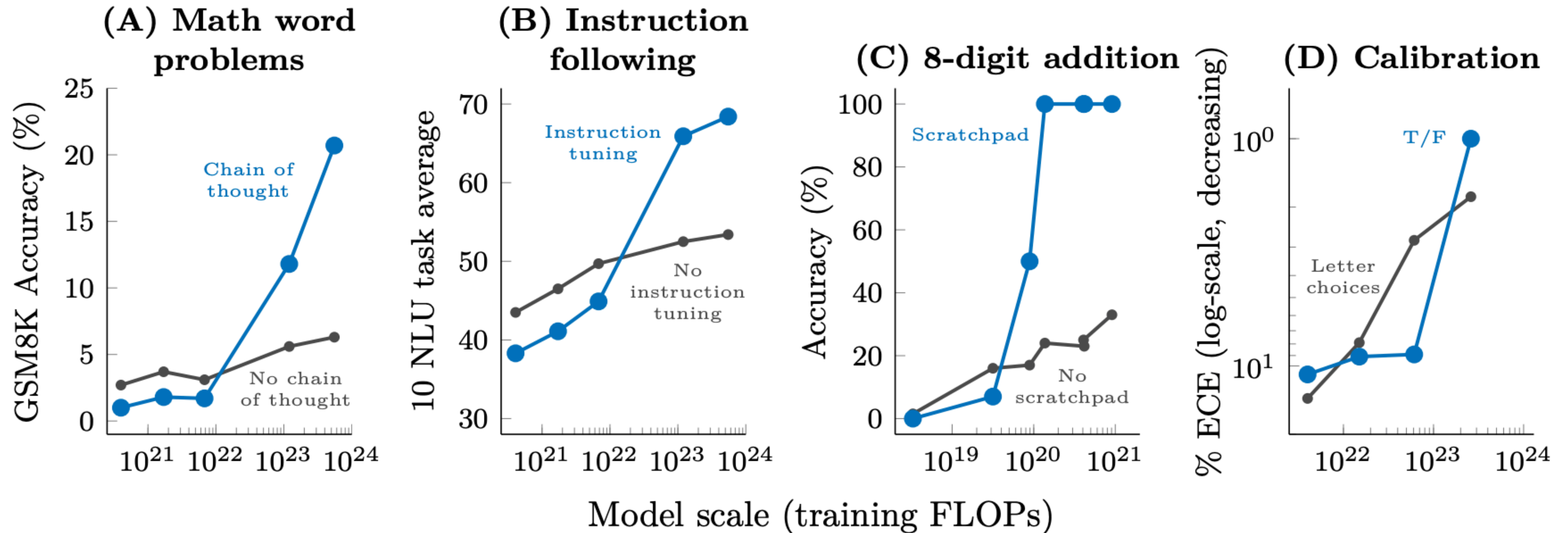
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

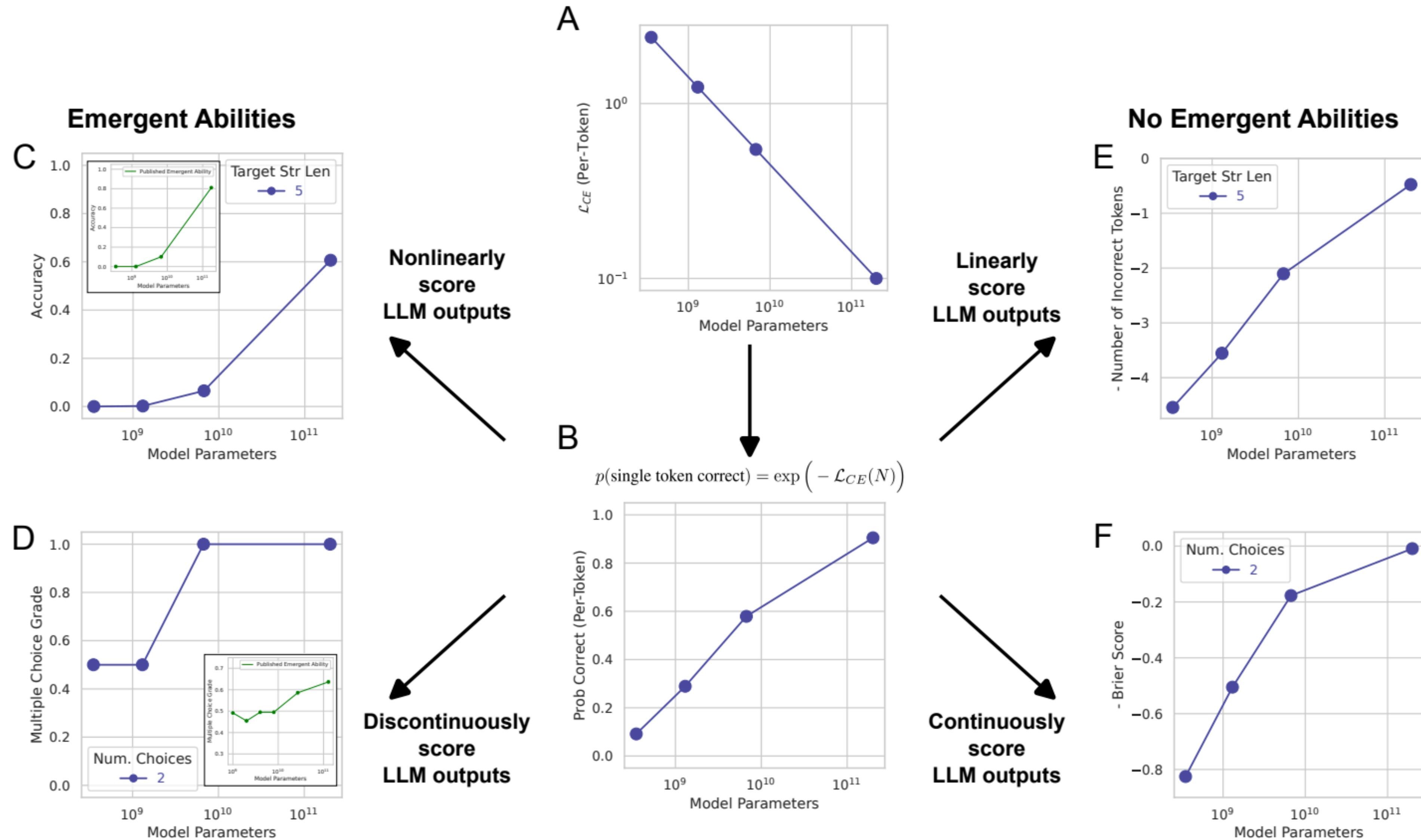
A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-thought 'reasoning' as an emergent behavior



With in-context learning and chain-of-thought reasoning, it's useful to reflect on (hard and soft) inductive biases for generalization.

Aside: Is it really a sudden jump? Or an artifact of our eval metrics?



Suppose scaling laws hold indefinitely. Would be awesome, right?

for each test problem. Unsurprisingly, we see that the 175B model significantly outperforms the smaller models. Assuming a log-linear trend, we can naively extrapolate these results to estimate that a model with 10^{16} parameters would be required to reach an 80% solve rate, when using the full GSM8K training

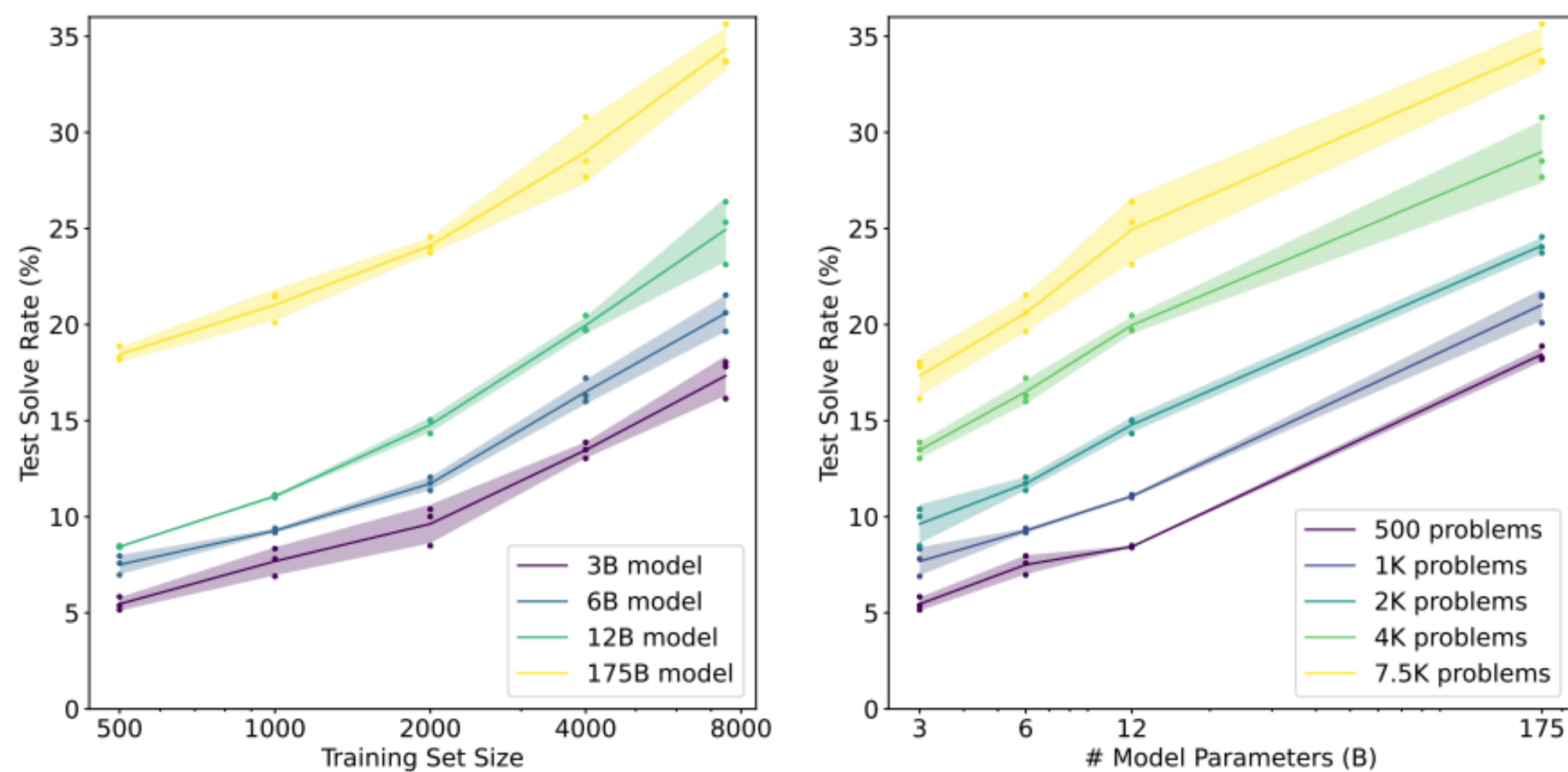


Figure 2: Final test performance for various GPT-3 model sizes after finetuning on training sets of different sizes. Mean and standard deviation is shown across

Even if scaling laws hold forever and we secure
infinite compute for **extremely large models...**

A photograph of Ilya Sutskever, a man with a beard and short hair, wearing a light green t-shirt and a purple lanyard. He is standing at a podium with a microphone, looking towards the right side of the frame.

Pre-training as we know it will end

Compute is growing:

- Better hardware
- Better algorithms
- Larger clusters

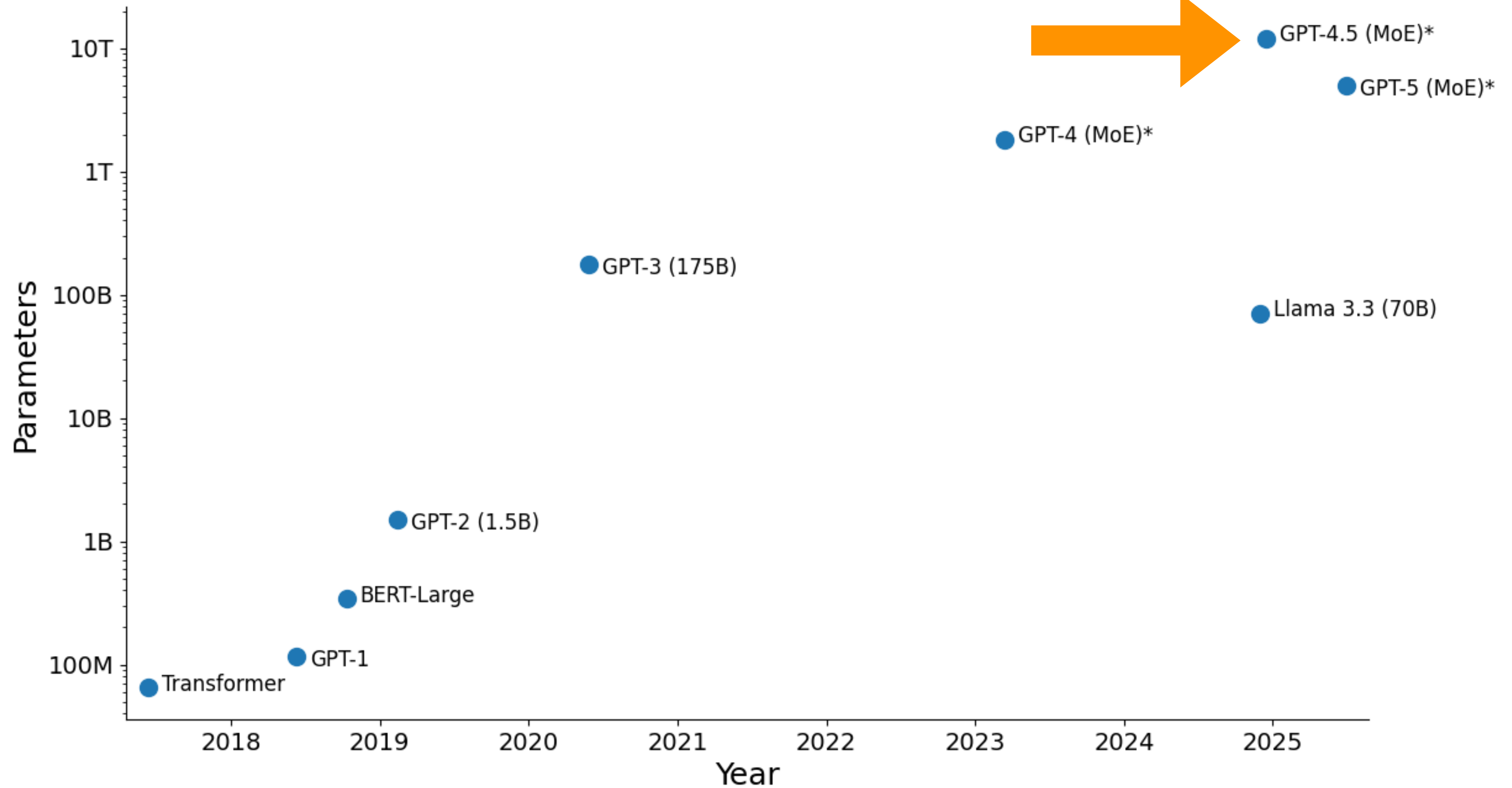
Data is not growing:

- We have but one internet
- The fossil fuel of AI

*NeurIPS'24
Keynote, by
Ilya Sutskever*

*So what's after
pre-training?*

Model Size Over Time



Very approximate (and speculative*) plot generated, with lots of iteration, by GPT-5. *For approximate trends only :-)*

If the loss is *low enough*, can we **more actively nudge the skills we want** instead of waiting for them to “emerge”?

Model	Score (ballpark)
GPT-3 175B (2020)	16%
PaLM 504B (2021)	18%
GPT-3 175B (+CoT)	47%
GPT-3 175B (+fine-tuning +verifier)	55%
Llama-2 70B (2023)	54%
Llama-3 8B (2024)	80%
Qwen 2.5 3B (2024)	87%
Qwen 3 0.6B (2025)	GSM8k even reported; too easy now!



Reuters

My News



Google clinches milestone gold at global math competition, while OpenAI also claims win

By Kenrick Cai and Jaspreet Singh

July 22, 2025 6:46 AM EDT · Updated July 22, 2025

*Post-training is a much more
“interesting” and human-
driven stage!*

(What’s a Bitter Lesson perspective?)

Some takeaways... is “scale all you need”?

- Good computational ideas **scale** and **scale well**. (Is power law good scaling?)
- Make no mistake: the original “make *parameter count* bigger, scale will just work out” is **essentially dead already**.
- Large *enough* models (much larger than you’d think!) appear to be **necessary**^(??) but far from **sufficient** or the primary axis for improvement!
- You *can* do much better by: scaling pre-training *data*, designing post-training, and leveraging smarter inductive biases and inference *systems* (e.g., reasoning & tool use). **See you in Lectures 15 and 21.**